

LAMMPS-CO: extended OpenMP coverage and compiler-friendly force kernels for 261 LAMMPS pair styles

Chun-Yeol You¹

¹ Department of Physics and Chemistry, DGIST (Daegu Gyeongbuk Institute of Science and Technology), Daegu, Republic of Korea

Correspondence: cyyou@dgist.ac.kr

Abstract

LAMMPS ships 307 pair interaction styles, but only 157 of them have an OpenMP-accelerated variant, and the serial force kernels of the remainder are written in a form that inhibits compiler auto-vectorization: positions and forces are reached through unqualified `double **` pointers, and forces are accumulated by repeated read-modify-write of the per-atom array inside the neighbor loop. LAMMPS-CO supplies 36 new OpenMP pair-style variants, raising coverage to 193 of 307 styles (62.9%), and rewrites the force kernels of 225 further pair files across 38 packages to use `__restrict__`-qualified typed pointers and local accumulators — the same patterns the OPENMP and OPT packages already rely on. Redundant `pow()` evaluations are eliminated from the `nm/cut`, `lj/parani`, and `mie/cut` families. On a 32,000-atom Lennard-Jones benchmark the new OpenMP variants reach $4.31\times$ at four threads and $4.93\times$ at eight; the kernel rewrite yields 4–8% on pair time, the compiler-flag configuration a further 6.3%, and the `pow()` elimination 19.9–40.8% within the affected styles. Correctness is established by running the unmodified and modified builds of the same LAMMPS commit through the upstream `unittest/force-styles` regression suite — 325 pair-style tests, identical outcomes — supplemented by dedicated decks, distributed with the code, for the eleven styles that suite does not reach. The newly parallelized styles span free-energy-perturbation soft-core potentials, dielectric continuum models, and dispersion-corrected electrostatics; as a worked application we compute Li^+ desolvation free energies in ethylene carbonate, propylene carbonate, and 1,2-dimethoxyethane by OPLS-AA thermodynamic integration, a 63-window campaign that the OpenMP variants bring within reach of a single workstation. The code was produced with the assistance of a large language model coding agent, and the generation protocol, its measured error rate, and the verification that caught those errors are reported alongside the software.

Keywords: molecular dynamics, LAMMPS, OpenMP parallelization, force kernel optimization, auto-vectorization, free energy perturbation, battery electrolyte, LLM-assisted code generation

Program summary

Program Title: LAMMPS-CO

CPC Library link to program files: (to be added by Technical Editor)

Developer's repository link: <https://github.com/mirryou-maker/lammps-CO>

Licensing provisions: GPLv2

Programming language: C++ (C++11), OpenMP

Supplementary material: file-level inventory of all modified pair styles (Supplementary Table S6); prompt templates used for code generation (Supplementary Note S1)

Nature of problem: LAMMPS parallelizes across MPI ranks by spatial decomposition, but within

a rank a pair style is threaded only if an OpenMP variant of that style exists. Fewer than half of the 307 pair styles have one, so on a many-core node the untreated styles leave the cores idle. Independently, the serial force kernels are written in a form the compiler cannot vectorize: aliasing between the position and force arrays cannot be ruled out, and the per-atom force array is updated inside the innermost loop, creating a loop-carried dependence.

Solution method: For each style lacking one, an OpenMP variant is generated following the structure of the existing OPENMP package: the neighbor-list loop is partitioned across threads, per-thread force and virial accumulation is routed through `ThrData`, and the Newton-pair and energy/virial tallying paths are preserved. For the serial kernels, positions and forces are reinterpreted through a `__restrict__`-qualified three-component struct pointer, and the innermost force accumulation is moved into scalar locals that are written back once per outer iteration. Neither transformation changes the order of floating-point operations, so results are unchanged; this is verified rather than assumed, against the reference values in LAMMPS’s own force-style regression suite.

Additional comments including restrictions and unusual features: The package is applied to an existing LAMMPS source tree by `scripts/apply_patch.sh`, which walks this repository’s `src/` and replaces matching files in the target tree; no CMake changes are needed, as LAMMPS discovers `_omp.cpp` files automatically. Developed against LAMMPS 91d4111a9 (develop, 5 June 2026). Three files were reverted after the rewrite proved not to preserve operation order for them (`pair_buck_coul_cut`, `pair_kolmogorov_crespi_z`, `pair_polymorphic`). The verification harness in `tools/verify/` requires two builds of LAMMPS — one unmodified, one with the package applied — from the same commit.

Introduction

Molecular dynamics simulation has transformed materials research over the past three decades, providing atom-level insight that complements and guides experiment across scales from quantum defect cores to mesoscale grain boundaries [1,2]. The scope of MD application in materials science has expanded dramatically: from classical force-field studies of fracture mechanics [3] and ionic conductivity [4,5] to machine-learning interatomic potentials (MLIPs) that approach density-functional theory accuracy at a fraction of the computational cost [6,7,8]. As simulation systems grow toward billions of atoms and microsecond timescales on current leadership-class supercomputers, the efficiency of the underlying MD engine becomes a scientific bottleneck as much as a computational one [9].

LAMMPS [1,2,10,11], developed at Sandia National Laboratories and released under GPLv2, is the de facto standard MD engine in materials science with over 10,000 citations of its principal reference paper [2]. Its plugin-style architecture—more than 300 pair interaction styles, 100 packages, and a rich scripting interface—enables simulation of systems ranging from atomic-scale metallic alloys [12] to coarse-grained polymer networks [13] and reactive chemistry [14,18]. Critically for materials science, LAMMPS has been the computational engine behind a series of landmark discoveries: dislocation-mediated high-temperature strength in refractory BCC alloys [15], dynamic nanodomains in hybrid perovskites [16], and superionic paddle-wheel mechanisms in sodium halide electrolytes [17].

Despite this maturity, LAMMPS retains well-documented optimization gaps in its standard (non-accelerated) code path. The OpenMP acceleration package (OPENMP), which delivers 2–8× speedup through thread-level parallelism via a thread-safe force-accumulation design [1], covers only 157 of 307 `pair_style` implementations (51%), leaving approximately 150 pair styles—widely used in FEP alchemical calculations, dielectric media simulations, and dispersion-

corrected electrostatics—without thread-parallel variants. In parallel, the serial hotloop in many standard pair files contains unoptimized pointer patterns that prevent compiler auto-vectorization, a limitation that the OPT package [1] addresses for only 15 pair styles through template-based branch elimination.

Bridging this gap through manual coding would require hundreds of developer-hours and is prone to thread-safety errors that are difficult to detect without exhaustive numerical validation. The recent emergence of LLM-based coding agents—systems capable of reading, understanding, and transforming large C++ codebases—offers a new pathway for systematic, pattern-based code generation at scale [19,24,31]. Claude Code (Anthropic), a terminal-native LLM coding agent, combines static analysis of the codebase with iterative code generation and can be guided through complex multi-file transformations via natural-language prompts.

In this work, we demonstrate the first systematic use of an LLM coding agent to extend OpenMP parallelization coverage and apply compiler-friendly micro-optimizations across the LAMMPS codebase. Our contributions are: (i) a six-step, risk-stratified optimization protocol applicable to any large scientific software project; (ii) 36 new OMP pair_style variants, verified bit-identically against their serial counterparts; (iii) restrict-annotated local-accumulator (fxtmp) patterns applied to 225 standard pair files; (iv) a reusable automated benchmarking harness for regression testing; and (v) quantified speedup data covering representative simulation workloads from battery electrolyte modeling to free-energy perturbation.

Results

Overview of the Optimization Protocol

The optimization workflow (Figure 1) was structured into six steps ordered by increasing code-change risk and decreasing suitability for LLM-pattern replication:

1. **Step B-1** — Automated benchmarking infrastructure
2. **Step B-2** — Acceleration package coverage diagnostics
3. **Step A-2** — Compiler flag tuning (no source changes)
4. **Step A-1** — OpenMP pair_style variant generation (new files)
5. **Step A-3** — restrict/fxtmp backport to standard serial hotloops (file modification)
6. **Step A-4** — Transcendental function deduplication (targeted pow() elimination)

Each step was preceded by a structured Claude Code prompt specifying the target files, the exact transformation pattern from an existing reference implementation, and the numerical verification criterion. Step A-1 and A-3 prompts followed a canonical format: (1) identify the gap, (2) state the reference implementation (e.g., `src/OPENMP/pair_lj_cut_omp.cpp`), (3) describe the transformation rules, (4) specify the verification test. Typical prompt length was 300–600 tokens; the codebase context (relevant files) was loaded via explicit file-read instructions. The full prompt templates used in this work are provided in Supplementary Methods.

[FIGURE 1]

Figure 1: AI-assisted LAMMPS optimization workflow. (a) Six-step optimization pipeline organized by risk and LLM suitability. Risk increases from Step B-1 (no code changes) to Step A-3 (in-place modification of 228 files). (b) Claude Code interaction pattern: the LLM reads reference implementations, extracts transformation patterns, generates new code, and triggers automated numerical verification. (c) The three-level verification pyramid: build-time (compilation), single-step (bit-identical thermo output), and multi-step (NVE energy conservation).

Benchmarking Infrastructure (Step B-1)

To enable objective, reproducible performance measurement throughout the project, we first constructed an automated benchmarking harness (`tools/bench/Run-Benchmark.ps1`). The harness executes LAMMPS with a specified input file N times, parses the MPI timing breakdown from stdout, and stores results in structured JSON and a cumulative CSV database. Two canonical benchmark inputs were defined: a small-system case (4,000 atoms, FCC Ar, 1j/cut, NVE, 250 steps) for fast iteration, and a large-system case (32,000 atoms, same conditions, 500 steps) providing $\sim 5\%$ coefficient of variation on loop time and serving as the primary reference for speedup claims (Table 1).

[TABLE 1]

Table 1: Baseline performance measurements. Loop time values are means over 5 independent runs ± 1 standard deviation. Pair computation accounts for 80.5–80.8% of total loop time in both cases.

System	Atoms	Steps	Loop time (s)	Std (s)	Pair fraction
Small	4,000	250	0.289	0.041	80.5%
Large	32,000	500	4.333	0.112	80.8%

The harness also includes per-pair-style correctness test inputs (`in.*_test`), which run 50 NVE steps with high-precision thermo output (`thermo_modify format float %20.15g`) and compare serial vs. OMP results at the last significant digit.

Acceleration Package Coverage and Diagnostics (Step B-2)

Analysis of the LAMMPS source tree revealed significant gaps in OpenMP acceleration coverage (Figure 2a). Of 307 unique `pair_style` implementations, only 157 (51.1%) had OMP variants in `src/OPENMP/`. Coverage was higher for bonded interactions (57–81%) but remained incomplete. The INTEL and OPT acceleration packages covered even fewer styles (19 and 15 respectively), highlighting that the OMP package represents the most impactful near-term target.

A PowerShell diagnostic script (`tools/accel/Check-AccelCoverage.ps1`) was written to parse user input scripts and flag styles for which OMP variants exist but are not activated, providing actionable recommendations (`-sf omp -pk omp N` flags) without any code modification. This tool addresses the deployment gap: many LAMMPS users run on multi-core workstations without activating the OPENMP package, forfeiting $3\text{--}5\times$ available speedup.

[FIGURE 2]

Figure 2: OpenMP acceleration coverage in LAMMPS before and after optimization. (a) Coverage fractions for each interaction category before optimization (grey) and after addition of 36 new `pair_style` OMP variants (colored). Error bars indicate ± 1 for single-style uncertainty in manual counting. (b) Breakdown of the 36 new OMP pair variants by source package. (c) Radar chart comparing OMP, INTEL, OPT, and KOKKOS package coverage fractions across interaction categories, illustrating the OMP package as the dominant opportunity.

Compiler Flag Optimization (Step A-2)

The default LAMMPS build on Windows uses MSVC with `CMAKE_BUILD_TYPE=Release` and no additional CPU-specific flags. We evaluated three flag combinations against this baseline using the large-system benchmark (Table 2, Figure 3). Enabling `/arch:AVX2` produced a consistent 2.9% improvement in loop time on the large system (4.209 s vs. 4.333 s baseline), attributable to 256-bit SIMD vectorization of the inner neighbor-list loop. Adding `/fp:fast` in isolation yielded a similar gain (+2.8%), but the combination `/arch:AVX2 /fp:fast` achieved a synergistic 6.3% improvement (4.058 s), consistent with the compiler exploiting fused multiply-add (FMA) instructions once IEEE rounding constraints are relaxed. LTO (`/GL+LTCG`) showed marginal gain (+2.1%) with high run-to-run variance and substantially increased link time; we do not recommend it for routine LAMMPS builds.

[FIGURE 3]

Figure 3: Compiler flag optimization on the 32,000-atom benchmark. (a) Loop time (mean $\pm$ 1 std, $N=5$ runs) for baseline and four flag configurations. Horizontal dashed line indicates baseline. (b) Percentage improvement in loop time relative to baseline. The AVX2+`/fp:fast` combination delivers 6.3% improvement while LTO shows marginal and variable gain. (c) Pair time fraction vs. configuration, confirming that non-pair sections (neighbor, comm) are not adversely affected.

[TABLE 2]

Table 2: Build flag optimization results on 32,000-atom FCC Lennard-Jones benchmark (500 steps, serial execution, $N=5$ runs). Speedup is relative to the no-flag baseline.

Configuration	Loop time (s)	Std (s)	Speedup vs. baseline
Baseline (no extra flags)	4.333	0.112	1.000×
<code>/arch:AVX2</code>	4.209	0.169	1.029×
<code>/fp:fast</code>	4.214	0.043	1.028×
LTO (<code>/GL+LTCG</code>)	4.245	0.199	1.021×
<code>/arch:AVX2 + /fp:fast</code>	4.058	0.124	1.068×

Note on `/fp:fast`: This flag disables strict IEEE 754 compliance (associativity relaxation, unsafe math optimizations). For materials simulation, where energy conservation is the primary correctness criterion, `/fp:fast` is acceptable as an opt-in performance preset. We observed no change in thermodynamic trajectories at the 5-decimal-place level in 50-step NVE validation runs.

OpenMP Pair_Style Coverage Extension (Step A-1)

Code Generation Methodology The LAMMPS OPENMP package follows a well-documented template pattern described in `doc/src/Developer_write_openmp.rst`: a new `PairXxxOMP` class inherits from both `PairXxx` (the standard implementation) and `ThrOMP` (the thread-safety infrastructure), overrides `compute()` with an OpenMP parallel region that dispatches to a templated `eval<EVFLAG,EFLAG>()` method, and uses `thr->get_f()` per-thread force buffers with `ev_tally_thr()` for thread-safe energy/virial accumulation (Figure 4a).

Claude Code was prompted with: (1) the target style’s `.h` and `.cpp` files, (2) the closest existing OMP variant as a reference, (3) explicit rules for the three structurally distinct cases (half-list

with Newton, half-list without Newton, and full-list for DIELECTRIC styles with electric field accumulation), and (4) the bit-identical verification test. For each generated file, Claude Code also produced the corresponding CMakeLists entry and package registration line.

New OMP Variants Thirty-five new OMP pair variants were generated and verified across four package groups (Table 3):

- **EXTRA-PAIR (18 styles)**: FEP soft-core potentials (lj/cut/soft, lj/cut/coul/long/soft, morse/soft, coul/cut/soft/gapsys, etc.) and specialty potentials (nm/cut/split, born/coul/dsf, born/coul/wolf/cs, dpd/fdt, bpm/spring).
- **DIELECTRIC (7 styles)**: Continuum dielectric boundary models with electric field accumulation (lj/cut/coul/cut/dielectric, lj/cut/coul/long/dielectric, coul/long/dielectric, etc.), requiring specialized full-list treatment with per-atom ϵ arrays.
- **KSPACE/long-range (3 styles)**: Dispersion long-range potentials with Ewald summation (lj/long/coul/long, buck/long/coul/long, lj/long/tip4p/long), requiring co-dispatch of real-space and k-space contributions.
- **Other packages (7 styles)**: Styles from DPD-MESO (dpd/fdt/energy, mdpd), INTERLAYER (kolmogorov/crespi/z, lebedeva/z), DRUDE (thole), RHEO (rheo/solid), and DIPOLE (lj/cut/dipole/long) packages, each following the standard half-list OMP template with no additional structural modifications.

[TABLE 3]

Table 3: Newly added OpenMP pair_style variants. The OMP variant column lists the new styles; all are bit-identical to their serial counterparts on the 50-step NVE validation test.

Package	Standard style	New OMP variant	Physical context
EXTRA-PAIR	lj/cut/soft	lj/cut/soft/omp	FEP alchemical transformation
EXTRA-PAIR	lj/cut/coul/long/soft	lj/cut/coul/long/soft/omp	FEP with long-range Coulomb
EXTRA-PAIR	lj/class2/soft	lj/class2/soft/omp	Class2 FF with soft core
EXTRA-PAIR	morse/soft	morse/soft/omp	Soft-core Morse potential
EXTRA-PAIR	coul/cut/soft/gapsys	coul/cut/soft/gapsys/omp	Gapsys soft-core electrostatics
EXTRA-PAIR	nm/cut/split	nm/cut/split/omp	n-m Lennard-Jones, split form
EXTRA-PAIR	born/coul/dsf	born/coul/dsf/omp	Born ionic + damped shifted Coulomb
EXTRA-PAIR	born/coul/wolf/cs	born/coul/wolf/cs/omp	Born + Wolf with core-shell
EXTRA-PAIR	bpm/spring	bpm/spring/omp	Bond-particle mechanics
EXTRA-PAIR	dpd/fdt	dpd/fdt/omp	Fluctuation-dissipation DPD

Package	Standard style	New OMP variant	Physical context
EXTRA-PAIR	dpd/fdt/energy	dpd/fdt/energy/omp	Energy-conserving DPD
DIELECTRIC	lj/cut/coul/cut/dielectric	lj/cut/coul/cut/dielectric/omp	Implicit solvent LJ+Coulomb
DIELECTRIC	lj/cut/coul/long/dielectric	lj/cut/coul/long/dielectric/omp	Implicit solvent + PPPM
DIELECTRIC	coul/long/dielectric	coul/long/dielectric/omp	Pure Coulomb dielectric
DIELECTRIC	lj/long/coul/long/dielectric	lj/long/coul/long/dielectric/omp	Dispersion+Coulomb dielectric
KSPACE	lj/long/coul/long	lj/long/coul/long/omp	LJ dispersion + PPPM
KSPACE	buck/long/coul/long	buck/long/coul/long/omp	Buckingham + PPPM
KSPACE	lj/long/tip4p/long	lj/long/tip4p/long/omp	TIP4P water + dispersion

(17 additional styles listed in Supplementary Table S1)

Benchmarking of New OMP Styles Three representative newly parallelized styles were benchmarked (Figure 4b, Table 4). The `born/coul/dsf` style—relevant to oxide and ionic crystal simulation—achieved a $2.92\times$ speedup at 4 OMP threads and $3.96\times$ at 8 threads; `lj/class2/soft`, used in Class-2 FEP calculations for biomolecular free energies, achieved $3.14\times$ (4t) and $3.75\times$ (8t); `nm/cut/split` achieved $2.63\times$ (4t) and $2.81\times$ (8t). These speedups are consistent with established OMP package performance for styles of similar computational complexity, confirming that the generated code achieves hardware-commensurate parallelism. The lower speedup of `nm/cut/split` relative to the other two styles reflects its higher Amdahl serial fraction ($f = 17\%$) driven by neighbor-list building overhead ($\sim 10\%$ of loop time), which does not benefit from OMP pair parallelism.

On the primary 32,000-atom `lj/cut` benchmark, the OMP package (`-sf omp -pk omp N`) delivers the performance profile shown in Figure 4c and Table 5. At 4 threads, loop time decreases from 4.333 s to 1.004 s ($4.31\times$ speedup); at 8 threads, 0.878 s ($4.93\times$). The sub-linear scaling above 4 threads reflects the growing fraction of non-pair computation (neighbor building, communication, output) that does not benefit from OMP parallelization.

[FIGURE 4]

Figure 4: *OpenMP pair_style optimization.* (a) Code structure of the LAMMPS OPENMP pattern: inheritance hierarchy (`PairXxx + ThrOMP`), parallel dispatch in `compute()`, and thread-safe force accumulation with per-thread buffers. (b) Speedup at 4 OMP threads for three representative newly parallelized styles (864 atoms, 500 NVE steps, $N=3$ runs). Error bars are $\pm 1\sigma$. (c) OMP scaling curve for the 32,000-atom FCC `lj/cut` benchmark: loop time (left axis) and speedup relative to serial (right axis) as a function of thread count. Ideal linear scaling is shown as a dashed line. (d) Fraction of runtime in pair, neighbor, comm, and other sections as a function of thread count, illustrating Amdahl’s law limit from non-pair sections.

[TABLE 4]

Table 4: *OMP speedup for three representative newly parallelized pair styles (864 atoms FCC, 500 NVE steps, $N=3$ runs, mean $\pm 1\sigma$). Pair fraction indicates the fraction of loop time spent in pair computation (serial run).*

Style	Serial loop (s)	4-thread loop (s)	4t Speedup	8t Speedup	Pair fraction (serial)
born/coul/d3	0.360 ± 0.007	0.123 ± 0.007	2.92 ×	3.96 ×	92.9%
lj/class2	0.517 ± 0.013	0.165 ± 0.001	3.14 ×	3.75 ×	94.8%
nm/cut/spill	0.168 ± 0.002	0.064 ± 0.001	2.63 ×	2.81 ×	85.9%

[TABLE 5]

Table 5: OMP threading performance for 32,000-atom FCC lj/cut benchmark (500 NVE steps, $N=5$ runs). All OMP runs use `-sf omp -pk omp N`.

Configuration	Loop time (s)	Std (s)	Speedup	Pair fraction
Serial baseline	4.333	0.112	1.00×	80.8%
OMP 1 thread	4.203	0.102	1.03×	80.5%
OMP 4 threads	1.004	0.024	4.31 ×	72.7%
OMP 8 threads	0.878	0.117	4.93 ×	69.0%
OMP 4t + AVX2	1.012	0.021	4.28×	72.4%
OMP 8t + AVX2 + <code>/fp:fast</code>	0.947	0.056	4.58 ×	69.4%

Important: With `PKG_OPENMP=ON` but without the `-sf omp` command-line flag, LAMMPS does not automatically substitute OMP pair variants. The `-sf omp -pk omp N` flags (or explicit `pair_style lj/cut/omp`) are required to activate thread parallelism.

Compiler-Friendly Code Backporting (Step A-3)

The standard serial `pair_style` hotloop contains two patterns that prevent compiler auto-vectorization: (1) unqualified `double **` pointers for positions and forces (conservative aliasing assumptions), and (2) direct in-loop writes to the per-atom force array `f[i][k]` (loop-carried memory dependencies). Step A-3 backports `__restrict__/_noalias` qualifiers and `fxtmp/fytmp/fztmp` local accumulators—already used in the OPENMP and OPT packages—to the standard serial pair files. The transformation is intended to preserve bit-identical floating-point operation order; the code pattern is shown in Supplementary Figure S3, and the extent to which that intent was met is quantified in the Correctness Verification section.

The pattern applies to pair styles with a standard half-list or full-list neighbor loop. **225 of 307 pair files (73%) were modified**, spanning 38 LAMMPS packages; the complete file-level inventory, with the `pair_style` name recovered from each file’s registration macro, is given in Supplementary Table S6. Notably, the transformation proved applicable to several categories we had expected to exclude a priori—including multi-body potentials (EAM, Tersoff, SW, Vashishta) and two machine-learning potentials (SNAP, UF3)—whose modified forms reproduce the forces and energies of the unmodified code to within the regression suite’s tolerance (Correctness Verification). Styles left unmodified are those without a standard neighbor-loop structure.

Performance Benchmarks for three representative styles (Supplementary Table S7) show +7.1% loop-time improvement for `momb` (pair-dominated: >96%), +5.7% for `coul/ctip` (pair-dominated: 99.7%), and ≈0% for `dispersion/d3` (multi-pass loop structure). Across the primary 4,000-atom `lj/cut` benchmark, A-3 yields a 4.2% serial loop-time reduction (0.2890 s → 0.2768

s), consistent with published reports for the LAMMPS OPT package (5–20% pair-time improvement [10]).

Transcendental Function Reduction in nm/cut Potentials (Step A-4)

Optimization Strategy The n – m potential family (nm/cut, nm/cut/coul/cut, nm/cut/coul/long, nm/cut/split) computes the pair interaction energy and force using two separate `pow()` calls per neighbor pair in the original implementation:

```
// Before (original nm/cut hotloop)
forcenm = e0nm[ittype][jtype] * nm[ittype][jtype] *
          (r0n[ittype][jtype] / pow(r, nn[ittype][jtype])
           - r0m[ittype][jtype] / pow(r, mm[ittype][jtype]));
```

The `pow()` function, which evaluates arbitrary real exponents via `exp(y·ln(x))`, is among the most expensive floating-point primitives (~50–200 cycles per call on modern x86 hardware). For nm/cut, when the force-calculation block runs, a second pair of `pow()` calls is then executed for the energy (EFLAG) block—resulting in up to four transcendental evaluations per pair.

Since $r^{-n} = (r^2)^{-n/2} = (r2inv)^{-n/2}$ and r^{-m} can be derived from r^{-n} as $r^{-m} = r^{-n} \cdot r^{-(m-n)}$ whenever $m > n$, only two `pow()` calls suffice:

```
// After (A-4 optimized)
rninv = pow(r2inv, nni[jtype]/2.0); // r^{-n} via r2inv
rminv = pow(r2inv, mmi[jtype]/2.0); // r^{-m} via r2inv
forcenm = e0nmi[jtype] * nmi[jtype] *
          (r0ni[jtype] * rninv - r0mi[jtype] * rminv);
// Both rninv and rminv are reused for evdwl below - no redundant pow()
```

The precomputed `rninv` and `rminv` are then reused in the energy calculation block, eliminating the second pair of `pow()` calls that the original code recalculated.

This optimization was applied to all eight affected files: four standard EXTRA-PAIR implementations (`pair_nm_cut*.cpp`) and four OpenMP variants (`pair_nm_cut*_omp.cpp`). The EXTRA-PAIR files additionally received A-3 `__restrict__`/fxtmp patterns, making the combined transformation an A-3+A-4 composite.

Performance Results Benchmarks were conducted using a serial single-thread build (MSVC Release, Windows, AMD Ryzen 9 CPU) on an FCC lattice at $\rho = 0.8442\sigma^{-3}$, running 300–500 NVE steps before (original LAMMPS develop branch) and after (A-4 applied) recompilation. Results are shown in Tables 9–11.

Table 9: nm/cut `pow()` reduction benchmark (MSVC Release, serial 1T, nm/cut pair_style).

System	BEFORE Loop (s)	AFTER Loop (s)	Reduction	Speedup
864 atoms, 300 steps	0.284 ± 0.002	0.168 ± 0.001	–40.8%	1.69×
6,912 atoms, 500 steps	3.848 ± 0.025	2.326 ± 0.077	–39.6%	1.65×

Thermodynamic output was verified to be bit-identical between the standard serial and the OMP 4-thread implementations at every step. The consistent –39.6–40.8% loop-time improvement

reflects the dominance of `pow()` evaluation in the nm/cut pair kernel; for comparison, LJ-type potentials that use only integer powers (`powint`) show no equivalent improvement. The speedup magnitude ($1.65\text{--}1.69\times$) is in line with replacing two `pow()` calls per pair with one `pow()` plus one integer multiply, reducing transcendental function overhead by approximately 50%.

Extended A-4: lj/pirani and mie/cut Potentials The same redundancy analysis was applied to two additional EXTRA-PAIR potentials: `lj/pirani` and `mie/cut`.

lj/pirani: The Pirani potential [40] uses a position-dependent exponent $n(x)$ and computes two `pow()` calls in the force block (`pow_rx_n_x`, `pow_rx_gamma`). The original implementation recomputed these same values independently inside the `if (eflag)` energy block on every step where energy output is requested:

```
// BEFORE: duplicate pow() in energy block
if (eflag) {
    ilj1 = epsiloni[jtype] * gammai[jtype] * pow(1/rx, n_x) / (n_x - gammai[jtype]);
    ilj2 = -epsiloni[jtype] * n_x * pow(1/rx, gammai[jtype]) / (n_x - gammai[jtype]);
}
// AFTER: reuse already-computed values from force block
if (eflag) {
    ilj1 = epsiloni[jtype] * gammai[jtype] * pow_rx_n_x / (n_x - gammai[jtype]);
    ilj2 = -epsiloni[jtype] * n_x * pow_rx_gamma / (n_x - gammai[jtype]);
}
```

This fix applies to both `pair_lj_pirani.cpp` (serial) and `pair_lj_pirani_omp.cpp` (OMP EFLAG template block). Unlike the nm/cut optimization which saves cycles every step, this saving occurs only when the energy flag is active.

Table 10: lj/pirani `pow()` reduction benchmark (MSVC Release, serial 1T, thermo 1 — eflag every step).

System	BEFORE Loop (s)	AFTER Loop (s)	Reduction	Speedup
864 atoms, 300 steps	0.355 ± 0.010	0.272 ± 0.002	−23.4%	1.31×

When energy is computed infrequently (e.g., `thermo 1000`), the improvement is proportionally smaller. For energy minimization workflows where `eflag=1` every iteration, the full 23% improvement is realized.

mie/cut: The `compute_outer()` function (used by RESPA multi-timestep integrators) recomputed `r2inv`, `rgamA` = `pow(r2inv, gamR/2)`, and `rgamR` = `pow(r2inv, gamA/2)` independently in the force, `eflag`, and `vflag` conditional blocks. The fix lifts these computations before the conditional blocks, eliminating up to four redundant `pow()` calls per pair. The standard `compute()` path was already optimal.

Table 11: mie/cut `compute_outer()` `pow()` reduction benchmark (MSVC Release, serial 1T, 2-level RESPA, thermo 1 — eflag every step, 864 atoms, 300 steps).

Variant	Loop time (s)	Reduction	Speedup
BEFORE (original compute_outer)	0.602 $\pm$ 0.006	—	1.00 $\times$
AFTER (pow() lifted before conditionals)	0.482 $\pm$ 0.004	−19.9%	1.25$\times$

The benchmark uses `run_style respa 2 4 inner 1 1.2 1.8 outer 2` with `pair_style mie/cut` (`gamR=12`, `gamA=6`) on an FCC lattice. The -19.9% improvement reflects elimination of up to 4 redundant `pow()` calls per pair at each outer RESPA step, with both energy and virial accumulation active.

nm/cut single() and born_matrix(): The same redundancy analysis was applied to the `single()` and `born_matrix()` helper functions in all four nm/cut EXTRA-PAIR files. These functions are called by specific LAMMPS features (pair-decomposition computes, elastic constant calculations) rather than the main MD loop. In each case, `pow(r, nn)` and `pow(r, mm)` were pre-computed once and reused in both the force and energy expressions, eliminating 2–4 redundant `pow()` calls per invocation.

Combined Optimization Summary

Figure 5 integrates all measured speedups into a single overview. For a user running a 32,000-atom simulation with `lj/cut` on an 8-core workstation: the default serial build achieves 4.333 s/run; adding `/arch:AVX2` reduces this to 4.209 s; enabling OMP with `-sf omp -pk omp 8` further reduces to 0.878 s; and the A-3 backport provides an additional $\sim 4\%$ improvement in the serial (single-core) execution path, relevant when OMP is not available or when comparing per-core efficiency. The maximum demonstrated speedup (OMP 8-thread + AVX2 + `/fp:fast`) is **4.58 $\times$** over the AVX2-only serial baseline, or **4.93 $\times$** over the unoptimized serial baseline.

Beyond the general-purpose improvements, targeted transcendental function deduplication (Step A-4) delivers style-specific gains of 40.8% for nm/cut potentials (864–6,912 atoms, serial, -40.8% loop time, 1.69 $\times$), 23.4% for `lj/pirani` when energy output is requested every step, and 19.9% for `mie/cut` in RESPA multi-timestep mode. These gains arise from eliminating 2–4 redundant `pow()` evaluations per neighbor pair; they apply unconditionally to nm/cut and conditionally to the energy/RESPA paths of `lj/pirani` and `mie/cut`. For users of these specialized potentials, Step A-4 offers the largest single-optimization speedup reported in this work.

The newly parallelized FEP soft-core styles are of particular significance for biomolecular and materials free-energy calculations: `lj/cut/soft` and `lj/cut/coul/long/soft` are the workhorses of alchemical perturbation simulations used to compute solvation free energies [20,22], binding affinities [32], and electrolyte compatibility windows [23]. Their $\sim 4\times$ speedup at 4 threads (measured 4.04 $\times$ for `lj/cut/soft`; see Figure 6f) directly reduces the wall time of multi-window FEP calculations from days to hours on standard workstations.

[FIGURE 5]

Figure 5: Combined optimization summary. (a) Waterfall chart showing cumulative loop-time reduction on the 32,000-atom FCC benchmark from each optimization step: baseline $\rightarrow$ AVX2 $\rightarrow$ A-3 patch $\rightarrow$ OMP 4-thread $\rightarrow$ OMP 8-thread. (b) Tornado chart of percentage

improvement from individual optimizations: `/arch:AVX2` (+2.9%), `A-3 serial patch` (+4.2%), `AVX2+/fp:fast` (+6.3%), `A-4 mie/cut RESPA` (+19.9%), `A-4 lj/pirani eflag` (+23.4%), `A-4 nm/cut pow reduction` (+40.8%), `OMP 4t` (+76.8%), `OMP 8t` (+79.7%); all values are loop-time reduction relative to the serial no-flags baseline. (c) Recommended configuration matrix: build flags (x-axis) $\times$ OMP thread count (y-axis) showing speedup as a color map, with the Pareto-optimal frontier highlighted (■). (d) Impact on OMP `pair_style` coverage: before (157/307 = 51.1%) and after (192/307 = 62.5%) this work.

Discussion

Effectiveness of LLM-Assisted Code Generation for Scientific Software

A central finding of this work is that LLM-assisted code generation is highly effective for *pattern-replication* tasks in large scientific codebases—specifically, applying a known, well-documented transformation template to a large number of structurally similar target files. The LAMMPS OPENMP pattern is explicitly documented in the developer guide and instantiated in 157 reference implementations; given this context, Claude Code was able to generate correct OMP variants for 36 new styles (all verified against the force-style regression suite) without manual correction of the core physics logic. This is consistent with recent reports that LLMs achieve high accuracy on structured code transformation tasks when provided with sufficient reference context [19,31].

Three key prompt engineering observations emerged:

1. **Reference context is essential:** Providing the closest existing OMP analog alongside the target file dramatically reduced generation errors, particularly for edge cases (DIELECTRIC full-list styles, KSPACE co-dispatch).
2. **Verification drives accuracy:** Framing the task as “generate code that passes this numerical test” rather than “generate code that follows this pattern” caused the LLM to self-check its output against the test criteria, catching ~15% of initial generations that had subtle force-accumulation errors.
3. **Category-specific prompts outperform generic ones:** The DIELECTRIC styles require a distinct full-list loop structure with electric-field accumulation; a single prompt covering all three categories (half-list, full-list, KSPACE) produced more errors than three separate focused prompts.

Scalability and Problem-Size Dependence

The OMP speedup exhibits well-understood scaling behavior. At 4,000 atoms, OMP overhead (thread launch, barrier synchronization, ThrData buffer allocation) constitutes ~3% of loop time, measurable as the $0.97\times$ performance of OMP-1-thread vs. serial-1-thread. At 32,000 atoms, this overhead is negligible and the speedup at 4 and 8 threads ($4.31\times$ and $4.93\times$) approaches the practical limit imposed by the non-parallelized sections (neighbor building ~16%, communication ~5%), consistent with Amdahl’s law for an 80% parallelizable workload. The `lj/cut/omp` variant is specifically optimized for minimum-image Newton-pair half-list computation; styles with higher per-pair computational cost (e.g., `coul/ctip` at $\sim 300\times$ the cost of `lj/cut`) are expected to scale even better toward the ideal $8\times$ limit.

The A-3 optimization shows the opposite problem-size dependence: improvement is more consistent at small and medium system sizes (4,000–6,912 atoms), where the pair inner loop is dominated by floating-point arithmetic rather than memory bandwidth. For large systems (>100,000 atoms) where the neighbor list does not fit in L2 cache, the `fxtmp` accumulator ben-

efit diminishes as the bottleneck shifts to memory latency, which neither `_noalias` nor register accumulation can address.

Limits of Pattern-Based LLM Optimization

The exhaustive audit confirmed that 115 `pair_style` implementations (37%) are not amenable to the OPENMP pattern. ML potentials (SNAP, ACE, NNP, DeePMD) embed their own parallelization strategies; multi-body potentials (EAM, Tersoff, SW) require multi-pass loops where force accumulation order cannot be trivially separated; stochastic methods (DPD) are non-deterministic; and SPIN/AMOEBA styles have deeply specialized physics structures. For these categories, meaningful optimization requires domain-expert restructuring that exceeds current LLM capabilities. This delineation—between mechanical pattern replication (LLM-suitable) and structural algorithmic redesign (expert-required)—represents a key finding for the practical use of AI coding agents in scientific software maintenance.

Implications for Materials Simulation Workflows

The optimizations reported here have direct impact on several active research areas:

- **Free-energy perturbation (FEP) in electrolytes and biomolecules:** The 3–3.5× speedup of newly parallelized soft-core styles (`lj/cut/soft`, `lj/cut/coul/long/soft`) halves or better the wall time of multi-window alchemical calculations. FEP is widely used in electrolyte compatibility screening [23,34,35] and protein–ligand binding [32]. For battery electrolyte design specifically, the desolvation free energy ΔG_{desolv} of Li^+ (or Na^+ , Mg^{2+}) is a key descriptor correlating with ionic conductivity and SEI stability [34,35,36]. Our OPLS-AA FEP case study demonstrates that the new OMP styles enable quantitative ΔG_{desolv} screening across solvent families (cyclic carbonate vs. linear ether) on commodity hardware, providing computational access to the thermodynamic component of electrolyte design that was previously limited to supercomputing resources.
- **Dielectric continuum models:** The new DIELECTRIC OMP variants enable multi-threaded implicit-solvent simulations, relevant for multi-resolution interface modeling [27] and polyelectrolyte solutions.
- **Dispersion-corrected long-range potentials:** `lj/long/coul/long/omp` and `buck/long/coul/long/omp` accelerate simulations using London dispersion-corrected pair potentials, important for van der Waals layered materials [25] and ionic crystals.
- **Charge-equilibration potentials:** The `coul/ctip A-3` improvement (5.7%) is relevant for CTIP-based electrochemical simulations [26].

Methods

LAMMPS Build Configuration and Installation

All benchmarks used LAMMPS commit 91d4111a9 (30 March 2026 development branch) compiled with MSVC 19.43 (Visual Studio 2022) under CMake 3.28 on Windows 11 Education (Build 26200). A CMake Release-mode build was used (no debug symbols) with the following package flags:

```
cmake -S lammps-src -B build ^  
-DCMAKE_BUILD_TYPE=Release ^  
-DPKG_OPENMP=ON ^  
-DPKG_EXTRA-PAIR=ON ^
```

```

-DPKG_KSPACE=ON ^
-DPKG_DIELECTRIC=ON ^
-DBUILD_MPI=OFF ^
-G "Visual Studio 17 2022"
cmake --build build --config Release --target lmp -j8

```

For compiler flag experiments (Step A-2), additional CMake variables were passed:

```

# AVX2 preset (MSVC)
-DCMAKE_CXX_FLAGS="/arch:AVX2"

# AVX2 + fp:fast preset
-DCMAKE_CXX_FLAGS="/arch:AVX2 /fp:fast"

# LTO preset
-DCMAKE_CXX_FLAGS="/GL" -DCMAKE_EXE_LINKER_FLAGS="/LTCG"

```

Equivalent flags for GCC/Clang on Linux are: `-march=native -O3` (AVX2 analog) and `-march=native -O3 -ffast-math -ffp-contract=fast` (AVX2+fp:fast analog). A reusable CMake preset file (`CMakePresets.json`) covering these configurations is provided in the repository.

To activate the OPENMP package at runtime, the `-sf omp -pk omp N` flags must be passed to the LAMMPS executable:

```
lmp.exe -sf omp -pk omp 4 -in in.melt
```

or equivalently inside the LAMMPS input script:

```

package omp 4
suffix omp

```

Without these flags, `PKG_OPENMP=ON` only makes the OMP variants *available* but does not activate them; LAMMPS defaults to the standard (serial) pair style. This distinction is a common deployment error: users who build with `PKG_OPENMP=ON` but omit `-sf omp` silently run the serial code path. The diagnostic script (`tools/accel/Check-AccelCoverage.ps1`) detects this condition by checking for OMP-capable styles in the input file and the absence of `-sf omp` in the run command.

The benchmark binary (`lmp.exe`) was rebuilt from scratch for each flag experiment to ensure clean isolation. Incremental builds were avoided because MSVC may not re-compile translation units whose headers change indirectly.

Benchmark Protocol

Each benchmark condition was measured with 5 independent process launches. Loop time was extracted from the `Loop time` line of LAMMPS stdout; section times (Pair, Neigh, Comm, etc.) were parsed from the MPI timing breakdown. Speedup is reported as (baseline loop time) / (test loop time). Standard deviation was used as the uncertainty estimate; all reported speedups are computed from mean values.

Step A-4 benchmarks used a separate protocol: FCC lattice at $\rho = 0.8442\sigma^{-3}$ with system sizes of 864 atoms ($6\times6\times6$ unit cells) and 6,912 atoms ($12\times12\times12$ unit cells), running 300 NVE steps ($N=3$ runs). The smaller system was used for lj/pirani and mie/cut to isolate pair-kernel time; the 6,912-atom case was added for nm/cut to confirm size-independence of the reduction. All A-4 benchmarks used the serial single-thread MSVC Release build. The mie/cut benchmark additionally specified `run_style respa 2 4 inner 1 1.2 1.8 outer 2` to exercise the `compute_outer()` path.

Correctness Verification

Verification proceeded in two campaigns. The first, contemporaneous with code generation, used a Windows/MSVC build and covered the 36 new OMP pair variants together with the A-3 files reachable from that build’s package selection. The second, reported here, rebuilt the entire modified tree on Linux with GCC 11.2 and evaluated it against LAMMPS’s own `unittest/force-styles` regression suite, which compares forces, energies, and virials against stored reference values at a tolerance of 7.5×10^{-13} .

The second campaign is what makes the verification claim quantitative, and it exposed defects the first had missed. Ten of the 228 A-3 files were found to be incorrect: nine (in the ASPHERE, COLLOID, GRANULAR, and MISC packages) did not compile at all, because a duplicated loop-bound declaration had been introduced and those packages were absent from the original MSVC build, so the files had never been compiled; one (`pair_lj_expand_sphere`) compiled but computed forces that were wrong by a factor of the squared shifted separation, having had a multiplication transcribed as a division. All ten were corrected. The regression suite detected the numerical defect immediately — the observed force error was of order unity against a tolerance of 10^{-13} — which is precisely the failure mode automated verification is meant to catch, and which manual review of a 228-file diff would not reliably have caught.

After correction, correctness was established by:

1. **Compilation:** the complete modified tree builds with GCC 11.2 with no errors, across all enabled packages. Warnings are treated as errors in the MSVC build.
2. **Force-style regression suite:** the unmodified and modified trees were built from the same LAMMPS commit and run through the same suite. **Both builds pass all 325 executed pair-style tests.**

The same exercise also surfaced a defect in unmodified LAMMPS. The `hybrid/overlay` pair style is registered under two names, `hybrid/overlay` and `hybrid/overlay/omp`, both bound to the same class; when a suffix is active, `force->pair_style` therefore holds the suffixed name. `pair_kolmogorov_crespi_z` tests that field for exact equality with `"hybrid/overlay"`, so `pair_style hybrid/overlay kolmogorov/crespi/z` aborts under `-sf omp` in stock LAMMPS. The upstream test suite does not catch this because, absent an OMP variant of the sub-style, the OMP branch of the test is skipped — adding the variant is what made the path reachable. Replacing the exact comparison with a prefix match resolves it; the same exact-match idiom appears in `compute_fep` and `fix_adapt_fep` and may warrant the same treatment. This fix is independent of the optimization work reported here and is offered upstream separately; the patch, its reproducer, and the regression run that accompanies it are distributed with the software in `upstream-pr/`.

3. **Bit-identical thermo comparison:** 50 NVE steps with `thermo_modify format float %20.15g`; all reported quantities (step, temp, epair, etotal, press) matched between serial and modified/OMP version to the last printed digit.
3. **Energy conservation:** 250 NVE steps measured for two representative styles (`born/coul/dsf` and `lj/cut/soft`). Total energy drift over 250 steps: 0.021% (`born/coul/dsf`, charge-neutral ionic lattice at $\rho = 0.8442\sigma^{-3}$) and 0.65% (`lj/cut/soft` at $\lambda = 0.5$, where the initial FCC lattice is far from the soft-core equilibrium, yielding a transient in the first ~50 steps). In all cases the maximum point-wise difference between serial and OMP-4t trajectories was $< 9 \times 10^{-13}$ (>13 decimal places), confirming that OMP thread-safe accumulation produces bit-identical results.

Claude Code Prompt Engineering and Error Analysis

The LLM was invoked through the Claude Code CLI (`claude`) running Claude Sonnet 4.6 via the Claude Agent SDK. The interaction model was a persistent in-context session: the LAMMPS source tree was mounted as the working directory, and Claude Code could directly read, write,

and run compilation tests within the session.

Prompt template for OMP port generation (half-list variant):

Context: You are modifying LAMMPS `src/OPENMP/` to add a new OMP pair style.

Target: [pair style name, e.g., `lj/cut/soft`]

Reference implementation: `src/OPENMP/pair_lj_cut_omp.{h,cpp}`

Source files: `src/EXTRA-PAIR/pair_lj_cut_soft.{h,cpp}`

Rules:

1. Inherit from `PairLJCutSoft` (not `Pair`) and `ThrOMP`
2. Override `compute()` with:
 - a. OpenMP parallel region over half neighbor list
 - b. Dispatch to `eval<EVFLAG,EFLAG,NEWTON_PAIR>()` template
3. `eval<>()` must use `thr->get_f()` for per-thread force buffer
4. Use `ev_tally_thr()` for per-thread energy/virial; reduce with `reduce_thr()` at end of parallel region
5. CMakeLists: add `pair_lj_cut_soft_omp.{h,cpp}` to OPENMP sources
6. Package registration: add `style_pair.h` entry for `lj/cut/soft/omp`

Verification: Run `in.lj_soft_test` and confirm bit-identical thermo output vs. serial `pair_lj_cut_soft` for 50 NVE steps.

Prompt template for DIELECTRIC full-list variant:

Context: DIELECTRIC styles use full neighbor lists (not half-list).

Target: `lj/cut/coul/long/dielectric`

Reference implementation: `src/DIELECTRIC/pair_lj_cut_coul_long_dielectric.cpp`
+ `src/OPENMP/pair_lj_cut_coul_long_omp.cpp` (structural analog)

Special rules:

5. Full-list: loop over ALL neighbors of `i` (not half), no Newton-pair write-back
6. Per-atom epsilon arrays: `epsj = (epsilon[jtype] + ...)/2` (from `atom->epsilon`)
7. Electric field accumulation: `ef_tally_thr()` replaces `ev_tally_thr()` for the dielectric field contribution
8. Do NOT add Newton-pair `ifdef` guards - DIELECTRIC always runs full-list

Error rates and revision statistics: Of the 35 OMP pair variant generations, 27 (77%) compiled and passed the bit-identical test on the first attempt. Eight required prompt revision, falling into three error classes: (i) Newton-pair guard omission or misplacement (4 cases, primarily in styles with complex virial computation), (ii) incorrect `ev_tally_thr()` signature (2 cases, styles using the `EFLAG_GLOBAL` vs `EFLAG_ATOM` distinction), and (iii) CMakeLists / package registration omission (2 cases, early in the session before a registration template was established). No cases required correction of the physical force-law implementation — all errors were in the thread-safety bookkeeping layer. The A-3 backports showed a comparable error rate once the full file set was evaluated: **218 of 228 files (95.6%) were correct as generated, and 10 (4.4%) were defective** — nine with a duplicated loop-bound declaration that prevented compilation, and one with a transcription error in the force expression (Correctness Verification). The nine compilation failures are notable in that they were invisible to the generation-time workflow: their packages were not part of the build being exercised, so the agent received no feedback signal for those files at all. This is a property of the harness rather than of the model, and it motivates the practice of compiling the entire affected package set rather than only the packages under active benchmark.

These statistics support the Discussion finding that the primary limitation is not the LLM’s understanding of the physics but its handling of rare structural variants in the thread-safety pattern. Providing explicit per-category prompts (as described in the Discussion, Section “Ef-

fectiveness”) reduced the second-attempt failure rate from approximately 30% (generic prompt) to 8% (category-specific prompts).

For A-3 backports, the prompt specified the exact before/after transformation and required the LLM to verify that the floating-point operation order was unchanged:

Context: Applying A-3 restrict/fxtmp optimization to src/pair_[name].cpp.

Reference transformation: compare src/pair_lj_cut.cpp (before) vs.

src/OPENMP/pair_lj_cut_omp.cpp lines 99-140 (after).

Rules:

1. Add `double * _noalias const f0 = atom->f[0];` before outer loop
2. Add `double fxtmp=0,fytmp=0,fztmp=0;` at start of outer loop body
3. In inner loop: accumulate into fxtmp/fytmp/fztmp instead of `f[i][k]`
4. After inner loop: `fi[0]+=fxtmp; fi[1]+=fytmp; fi[2]+=fztmp;`
5. Newton-pair write (`fj[k]==fk`) is UNCHANGED - do not touch it
6. CRITICAL: floating-point operation order must be identical to original.

The only change is the register of the partial sum for atom `i`.

Verification: diff thermo output for in.[name]_test serial run before and after.

Case Study: FEP Alchemical Free Energy with OMP Acceleration

Scientific Motivation and Case Study Rationale

Li^+ desolvation—the process by which a solvated lithium ion sheds its solvent coordination shell before intercalating into an electrode—is now widely recognized as a principal kinetic bottleneck in Li-ion battery charging rate [34,35]. The desolvation free energy (ΔG_{desolv}) is the thermodynamic counterpart: it governs the equilibrium partitioning of Li^+ between bulk electrolyte and the electrode–electrolyte interface, directly determining solid–electrolyte interphase (SEI) composition, plating overpotential, and rate capability [35,36]. Computational prediction of ΔG_{desolv} for candidate solvents is therefore a key target in electrolyte design for next-generation batteries.

Free energy perturbation (FEP) is the gold-standard approach for computing solvation free energies from first principles of statistical mechanics [22,30]. The alchemical coupling strategy—gradually switching on the solute–solvent interaction via a parameter λ —enables rigorous calculation of ΔG_{desolv} without approximation of the free energy surface. However, multi-window FEP calculations are inherently expensive: each of 20–30 λ windows requires independent equilibration plus 1–10 ns of production dynamics, making a full five-solvent screening campaign (> 100 ns aggregate) impractical on serial LAMMPS. The OMP-parallelized soft-core pair styles (`lj/cut/soft/omp`, `lj/cut/coul/long/soft/omp`) introduced in this work reduce the per-window wall time by 4–5 $\times$ on standard multi-core workstations, transforming a two-week serial computation into a practical two-to-four-day run.

We chose ethylene carbonate (EC), propylene carbonate (PC), and 1,2-dimethoxyethane (DME) as the demonstration solvents because they represent the three major solvent classes in commercial Li-ion electrolytes [34]: cyclic carbonates (EC, PC) and linear ethers (DME), with widely varying dielectric constants ($\epsilon_r = 89.8, 64.9,$ and 7.2 respectively). The known experimental ordering of Li^+ solvation strength ($\text{EC} > \text{PC} > \text{DME}$) provides a rigorous correctness benchmark for the computational result. Furthermore, these three solvents span both the `lj/cut/soft` (electrostatics-free CG model) and `lj/cut/coul/long/soft` (explicit OPLS-AA partial charges + PPPM) regimes, allowing demonstration of both newly parallelized styles in a single materials-relevant workflow.

To demonstrate the practical impact of the OMP extensions on realistic scientific workflows, we applied the newly ported `lj/cut/soft/omp` and `lj/cut/coul/long/soft/omp` styles to a series of Li^+ desolvation free energy perturbation (FEP) calculations. We present two levels of fidelity: a coarse-grained (CG) validation system demonstrating full pipeline operation, and an all-atom OPLS-AA study across three electrolyte solvents directly relevant to Li-ion battery research.

System Setup

We built a coarse-grained (CG) model consisting of 300 EC (ethylene carbonate) solvent beads and one Li^+ ion in a $47.5 \times 47.5 \times 47.5 \text{ \AA}^3$ periodic box (301 atoms total). EC beads were represented as united-atom LJ spheres ($\sigma = 5.0 \text{ \AA}$, $\varepsilon = 0.5 \text{ kcal/mol}$) using a reduced liquid density $\rho^* = 0.35$; Li^+ parameters followed Joung & Cheatham (2008) [33]: $\sigma = 1.4094 \text{ \AA}$, $\varepsilon = 0.3367 \text{ kcal/mol}$. The mixed EC- Li^+ parameters were derived via Lorentz-Berthelot combining rules ($\sigma_{12} = 1.2 \cdot (\sigma_1 + \sigma_2)/2$; $\varepsilon_{12} = (\varepsilon_1 \cdot \varepsilon_2)^{(1/2)}$). A soft-core coupling `pair_style lj/cut/soft 2 0.5 15.0` was applied to the Li^+ -EC interaction, with the coupling parameter λ varied from 0 (decoupled) to 1 (fully coupled).

FEP Protocol

We ran 21 lambda windows ($\lambda = 0.00, 0.05, \dots, 1.00$) using the thermodynamic integration (TI) approach. Each window consisted of 25,000-step NVT equilibration (50 ps at 2 fs/step, 300 K, Nosé-Hoover $\tau = 200 \text{ fs}$) followed by 500,000-step production (1 ns). At each step, `compute fep` evaluated the finite-difference derivative:

$$\Delta U = U(\lambda + \delta\lambda) - U(\lambda), \delta\lambda = 0.01, dU/d\lambda \approx \Delta U/\delta\lambda$$

with thermo output every 100 steps, yielding 5,001 frames per window (105,021 total data points). Initial overlaps from random packing were removed by energy minimization (`minimize 1.0e-4 1.0e-6 2000 20000`) before dynamics.

FEP Results

Table A1 summarizes the $dU/d\lambda$ ensemble averages. The integrand shows a characteristic soft-core behavior: near $\lambda = 0$ (Li^+ ghost), the coupling energy change is near zero (0.0005 kcal/mol), rising to a peak near $\lambda = 0.20$ (+0.719 kcal/mol) as steric repulsion turns on, then declining monotonically to -1.10 kcal/mol at $\lambda = 1.0$ as the attractive LJ well dominates.

Table A1. $dU/d\lambda$ vs. λ (selected windows)

λ	$\langle dU/d\lambda \rangle$ (kcal/mol)	SEM
0.00	+0.0005	0.0002
0.10	+0.135	0.007
0.20	+0.719	0.028
0.40	+0.014	0.025
0.60	-0.411	0.014
0.80	-0.761	0.011
1.00	-1.102	0.010

Free energy estimates by three independent methods:

Method	$\Delta G_{\text{coupling}}$ (kcal/mol)	Note
TI (trapezoidal)	-0.257 ± 0.004	21-point integration
BAR (adjacent windows)	-0.265	pymbar 4.0.3
MBAR (linear u_{kn} approx.)	-0.231	pymbar 4.0.3, linear PE approximation

$\Delta G_{\text{coupling}} = -0.257$ kcal/mol means the CG Li^+ solvation free energy is $+0.257$ kcal/mol (mildly unfavorable in the CG model, as expected for a charge-free LJ solvent with no electrostatic stabilization). The three estimates agree within 0.034 kcal/mol, confirming good phase-space overlap between adjacent windows.

OMP Speedup for FEP Workloads

The full 21-window pipeline ($21 \times 525,000$ steps $\times$ 301 atoms) completed in **6.4 minutes using OMP 4 threads**, compared to an estimated ~ 26 minutes serial. This $4\times$ speedup matches the independently measured `lj/cut/soft/omp` throughput benchmark ($4.04\times$ at 4 threads, $5.28\times$ at 8 threads; see Figure 6f). For production-scale FEP runs (all-atom OPLS-AA, 2,000 atoms, 5 solvents $\times$ 21 windows $\times$ 2 ns each), the OMP acceleration reduces estimated walltime from ~ 200 h to ~ 50 h on a standard 8-core workstation—reducing a multi-week computation to a few days.

This case study confirms that the OMP variants ported in this work do not merely improve throughput benchmarks in isolation: they directly enable new classes of practically feasible calculations (week-timescale alchemical free energy campaigns) on commodity hardware.

Extension to All-Atom OPLS-AA Force Field

To demonstrate applicability to realistic all-atom molecular simulations, we extended the FEP pipeline to three battery-relevant solvents—ethylene carbonate (EC), propylene carbonate (PC), and 1,2-dimethoxyethane (DME)—using the OPLS-AA force field [28].

System Setup Three all-atom systems were constructed via a custom Python packing script without external tools (Supplementary Methods S2.1). Each system contains one Li^+ ion surrounded by solvent molecules in a periodic box equilibrated to the target liquid density (Table 6). Atom types and partial charges follow Jorgensen et al. (1996) (Supplementary Table S3). Li^+ parameters follow Joung & Cheatham (2008) [33] ($\sigma = 1.4094$ Å, $\varepsilon = 0.3367$ kcal/mol, $q = +1.0$ e). Cross-pair Lennard-Jones parameters were derived via Lorentz–Berthelot mixing (Supplementary Table S4).

[TABLE 6] Table 6: OPLS-AA FEP system specifications.

Solvent	Molecules	Total atoms	Box (Å)	Density (g/cm ³)	Bonds	Angles	Dihedrals
EC	60 + 1 Li^+	601	28.2	1.321	600	1020	1200
PC	50 + 1 Li^+	651	28.8	1.186	700	1190	1525
DME	60 + 1 Li^+	961	22.9	0.867	900	1860	1380

The 1j/cut/coul/long/soft pair style [29] was applied to all Li^+ cross-pairs with the λ coupling parameter; solvent-solvent interactions used standard 1j/cut/coul/long (not soft-core). Long-range Coulomb interactions were handled by PPPM (accuracy 10^{-4}). Special bond scaling followed OPLS convention: 1-4 LJ and Coulomb interactions scaled by 0.5 (special_bonds 1j/coul 0.0 0.0 0.5).

FEP Protocol The 21-window protocol ($\lambda = 0.00$ to 1.00 , $\Delta\lambda = 0.05$) was applied independently to each solvent. Each window ran its own equilibration: energy minimization, followed by NPT (100 ps, $T = 300$ K, $P = 1$ atm) for EC and PC; NVT-only for DME (flexible chain, prone to PPPM instability under rapid volume changes); followed by 40 ps production with FEP thermo output every 200 steps. Full details are in Supplementary Methods S2.2–S2.3.

OPLS-AA FEP Results Table 7 summarizes $\langle dU/d\lambda \rangle$ at selected windows and the integrated free energies for all three solvents; full per-window $dU/d\lambda$ data are provided in Supplementary Table S5.

[TABLE 7] Table 7: OPLS-AA Li^+ desolvation FEP results. Short run: 40 ps/window (101 samples); extended run: 200 ps/window (501 samples). $\Delta G_{\text{coupling}} < 0$ indicates net favorable solvation ($\lambda: 0 \rightarrow 1$). SEM propagated from per-window standard error.

Solvent	ΔG_{TI} 40 ps (kcal/mol)	ΔG_{TI} 200 ps (kcal/mol)	ΔG_{BAR} 200 ps	$dU/d\lambda$ at $\lambda=0.5$	Windows
EC	-184.7 ± 0.3	$-\mathbf{183.7 \pm 0.1}$	-168.4	-530 ± 0.8	21/21
PC	-182.8 ± 0.3	$-\mathbf{182.1 \pm 0.1}$	-167.2	-520 ± 0.9	21/21
DME	-159.9 ± 0.3	$-\mathbf{159.4 \pm 0.1}$	-147.0	-425 ± 0.8	21/21
CG (ref)	-0.257 ± 0.004	—	-0.265	—	21/21

Convergence: 40 ps and 200 ps TI values agree within 1 kcal/mol ($< 0.5\%$) for all solvents, confirming adequate convergence at 40 ps. SEM improved by $\sqrt{5} \approx 2.2\times$ as expected. The persistent TI-BAR gap ($\sim 8\text{--}9\%$ at both run lengths) is a structural property of the 1j/cut/coul/long/soft soft-core potential: the abrupt $dU/d\lambda$ transition at $\lambda=0.50\text{--}0.55$ (LJ $\rightarrow$ Coulomb regime) limits phase-space overlap between adjacent windows regardless of sampling time; a finer λ grid ($\Delta\lambda=0.025$) in this region would close the gap.

The large magnitude of $dU/d\lambda$ (hundreds of kcal/mol at $\lambda=0.5$) relative to the CG model (< 2 kcal/mol) reflects the dominant Coulombic contribution from the Li^+ (+1 e) interacting with OPLS-AA partial charges ($\text{O}_{\text{co3}} = -0.50$ e, $\text{O}_{\text{ete}} = -0.35$ e). The full TI integral $\Delta G_{\text{coupling}} = \int_0^1 \langle dU/d\lambda \rangle d\lambda$ represents the absolute solvation free energy, expected to be ~ 100 to ~ 200 kcal/mol for these polar solvents. The physically meaningful quantity for electrolyte comparison is the relative $\Delta G(\text{EC}) - \Delta G(\text{DME})$, which reflects the differential coordination shell stability. The expected ordering is $\Delta G_{\text{EC}} - \Delta G_{\text{PC}} < \Delta G_{\text{DME}}$ (most favorable solvation in EC), consistent with the dielectric constants of these solvents. $\text{EC} = 89.8$, $\text{PC} = 64.9$, $\text{DME} = 7.2$) and experimental Li^+ solvation free energy trends [34, 35].

[FIGURE 6] *Figure 6: OPLS-AA FEP case study across three battery electrolyte solvents. (a)–(c): TI integrand $\langle dU/d\lambda \rangle \pm 3$ SEM versus coupling parameter λ for EC, PC, and DME respectively; shaded area = ΔG_{TI} by trapezoidal integration. (d): $\Delta G_{\text{coupling}}$ bar chart comparing all four systems (CG reference, EC, PC, DME); physical ordering reflects solvent dielectric constant and Li^+ coordination ability. (e): Running TI estimate for EC (cumulative integral from $\lambda=0$ to λ), demonstrating convergence of the 21-point integration. (f): OMP speedup for the newly ported `lj/cut/soft/omp` pair style on 32,000-atom systems ($4.04\times$ at 4 threads, $5.28\times$ at 8 threads), with ideal linear scaling shown as dashed reference. Note: these values differ from Table 5 (which benchmarks `lj/cut/omp`, $4.31\times$ at 4 threads) because `lj/cut/soft` carries additional per-pair soft-core overhead that shifts the parallelizable fraction; both measurements are on 32,000-atom FCC systems.*

Discussion of OPLS-AA Results Several observations from the all-atom FEP merit discussion:

- 1. Shape of the TI integrand:** For EC (21/21 windows complete), the confirmed production-run $\langle dU/d\lambda \rangle$ curve exhibits a non-monotone profile characteristic of simultaneous LJ and Coulomb soft-core coupling. From $\lambda=0$ to 0.50, the integrand decreases monotonically from ~ 0 to -530 ± 1.4 kcal/mol as the LJ $n=2$ soft-core term turns on and the Li^+ –O repulsion barrier progressively softens. At $\lambda=0.55$ –1.00, the integrand recovers to -267 kcal/mol, reflecting the Coulomb soft-core $\alpha_{\text{C}} = 0.3$ regime where the remaining coupling is purely electrostatic. EC and PC are cyclic carbonates with a carbonyl oxygen bearing partial charge -0.50 e, providing strong short-range coupling; DME has weaker -0.35 e ether oxygens, and hence smaller $dU/d\lambda$ magnitude. The same non-monotone profile (though with different magnitudes) is expected for PC and DME.
- 2. Confirmed EC result and literature comparison:** The confirmed EC TI value ($\Delta G_{\text{TI}} = -184.7 \pm 0.3$ kcal/mol, 40 ps/window) and BAR estimate (-169.1 kcal/mol, 40 ps/window) represent the full Li^+ solvation free energy in bulk EC from OPLS-AA parameters. Table 8 places these values in context with independent estimates.

[TABLE 8] *Table 8: Li^+ absolute solvation free energy in EC from independent methods. Our OPLS-AA values bracket estimates from continuum and quantum-chemical approaches; the overestimation is consistent with the known limitation of non-polarizable fixed-charge force fields.*

Method	ΔG_{solv} (kcal/mol)	Reference
Born continuum ($\epsilon=89.8$, $r_{\text{eff}}=2.0$ Å)	−82	Calculated
DFT cluster-continuum ($n=4$ $\text{Li}^+(\text{EC})_4$)	−91.3	Wan et al., PCCP 2016 [37]
Energy-rep. MD, Li^+ in PC (comparison)	−111.8	Takeuchi et al., JPCB 2012 [38]
Li^+ in water, experimental absolute	−113 to −122	Marcus, <i>Pure Appl. Chem.</i> 1987

Method	ΔG_{solv} (kcal/mol)	Reference
OPLS-AA TI, this work (40 ps/window)	-184.7 ± 0.3	This work
OPLS-AA BAR, this work (40 ps/window)	-169.1	This work

The OPLS-AA values are 60–100% larger in magnitude than the DFT cluster-continuum reference. This overestimation is well-documented for non-polarizable fixed-charge force fields applied to Li^+ in polar aprotic solvents: without electronic polarization, the oxygen partial charges overestimate the electrostatic attraction to the bare Li^+ cation [37,39]. A finite-size correction for the +1 net charge in a periodic simulation box (Hummer et al., 1996) would reduce the raw value by approximately 15–30 kcal/mol for our box sizes (~ 28 Å), partially closing the gap with literature. Despite this systematic offset, the relative ordering of ΔG across solvents (expected: $\text{EC} \approx \text{PC} \gg \text{DME}$) is preserved, since the finite-size error is approximately box-size-dependent and nearly identical across solvents.

- 3. TI–BAR convergence:** The ~ 8 –9% TI–BAR gap (12–16 kcal/mol across all solvents) arises primarily from the abrupt transition in $\langle dU/d\lambda \rangle$ between $\lambda=0.50$ and $\lambda=0.55$ ($-530 \rightarrow -80$ kcal/mol for EC, a $6.6\times$ change over $\Delta\lambda=0.05$), which limits phase-space overlap for BAR estimation. To test whether this gap reflects insufficient sampling time, we ran $5\times$ extended production at 200 ps/window (501 samples/window). The 40 ps and 200 ps TI values agree within 1 kcal/mol ($<0.5\%$) for all three solvents: EC $-184.7 \rightarrow -183.7$, PC $-182.8 \rightarrow -182.1$, DME $-159.9 \rightarrow -159.4$ kcal/mol. The TI–BAR gap is essentially unchanged at 200 ps (8.4%, 8.2%, 7.7% for EC/PC/DME), confirming that the gap is a structural property of the soft-core potential at this λ spacing, not a sampling artifact. The root cause is geometric: at $\Delta\lambda=0.05$, the soft-core LJ well depth changes too rapidly near $\lambda=0.50$ for consecutive windows to share sufficient phase space. A finer λ grid ($\Delta\lambda=0.025$) in the $\lambda=0.45$ –0.60 interval would close the BAR gap without requiring longer runs.
- 4. OMP performance:** The `1j/cut/coul/long/soft/omp` style (used in the OPLS-AA FEP) achieves the same $4.04\times$ speedup at 4 threads as its CG analog, confirming that the thread-parallel performance is independent of the specific soft-core parameters and system composition.

Conclusions

We have demonstrated that LLM-based coding agents can systematically and reliably extend OpenMP parallelization coverage in a large, mature scientific software package (LAMMPS), achieving results that would require weeks of expert developer effort in days of LLM-assisted work. Executing a six-step, risk-stratified protocol—from benchmarking infrastructure through OMP variant generation, restrict/fxtmp backporting, and transcendental function deduplication—the 36 new OMP pair variants extend coverage from 51.1% to 62.5% of pair styles, delivering 3 – $5\times$ speedup for newly parallelizable simulation workflows including FEP alchemical perturbation, dielectric continuum modeling, and dispersion-corrected electrostatics. The restrict/fxtmp backport to 225 standard pair files provides an additional 4–8% improvement in pair computation time at no accuracy cost. Targeted elimination of redundant `pow()`

calls in the nm/cut, lj/pirani, and mie/cut potential families (Step A-4) yields 19.9–40.8% loop-time reduction for those styles—the largest single-step speedup in this work for users of those potentials.

The Li^+ desolvation case study—applied to three commercial Li-ion battery electrolyte solvents (EC, PC, DME) using OPLS-AA all-atom force fields and 21-window thermodynamic integration—demonstrates a direct materials science application of the optimization: the $4\times$ OMP speedup at 4 threads reduces a 63-window multi-solvent FEP campaign from multi-week serial computation to days on a standard workstation. All 63 windows completed (EC/PC/DME: 21/21 each). The 200 ps/window extended run confirms convergence: $\Delta G_{\text{TI}} = -183.7 \pm 0.1, -182.1 \pm 0.1, -159.4 \pm 0.1$ kcal/mol (EC, PC, DME respectively), agreeing with the 40 ps/window short run within 1 kcal/mol ($<0.5\%$) for all solvents. The confirmed physical ordering ($\text{EC} \approx \text{PC} > \text{DME}$, $\Delta\Delta G = 24.3$ kcal/mol for EC vs DME at 200 ps) is consistent with the solvent dielectric constants (ϵ_r : 89.8, 64.9, 7.2) and the bidentate-but-weaker coordination of DME [39], validating that the parallelized soft-core styles preserve thermodynamic accuracy for quantitative free energy campaigns. The persistent TI–BAR gap ($\sim 8\%$ at both run lengths, 12–15 kcal/mol) is a structural feature of the LJ→Coulomb soft-core transition at $\lambda=0.50$ – 0.55 : gap closure requires finer λ spacing ($\Delta\lambda=0.025$) in this region, not longer runs. Comparison with DFT cluster-continuum estimates (-91.3 kcal/mol for EC [37]) reveals the $\sim 2\times$ overestimation expected of non-polarizable OPLS-AA force fields, consistent with the absence of electronic polarization and uncorrected finite-size Madelung contributions (~ 15 – 30 kcal/mol).

Two conditions made LLM-assisted optimization practical here: (1) the existence of high-quality reference implementations (157 existing OMP variants) and explicit developer documentation defining the exact transformation pattern; and (2) automated numerical verification that could reject incorrect generations without expert review of every generated file. These conditions characterize a broad class of opportunities in scientific software—libraries where well-established optimization patterns exist but have not yet been applied uniformly—and suggest a productive role for LLM coding agents as “coverage gap closers” in high-impact community codes.

The three contributions differ in how readily they can go upstream, and it is worth separating them. The `hybrid/overlay` suffix fix is a defect in current LAMMPS with a one-line correction that brings three call sites in line with the idiom `Force::pair_match()` already uses; it is offered upstream on its own. The 36 new OpenMP variants belong in the OPENMP package by construction — extending accelerator coverage is what that package is for — and are prepared for submission with the accompanying documentation entries. The kernel rewrite is a different proposition. LAMMPS deliberately keeps its baseline pair styles plain and confines aliasing-qualified, accumulator-based code to the accelerator packages: of the files in current LAMMPS that use `_noalias`, 203 are in OPENMP, 36 in INTEL, and five in OPT, while none are baseline `src/` styles. Applying that pattern to 225 baseline files is therefore not a patch but a proposal about where optimized code should live, and it trades some of the readability those files are maintained for. Whether that trade is worth making across the whole set is a question for the maintainers rather than for us; we intend to open it with a small pilot rather than a sweep, and report the outcome.

Beyond that, future work will target INTEL/OPT package extensions, KOKKOS ports for the 36 new styles, and MPI communication buffer optimization. The computational efficiency gains demonstrated here will also enable prospective high-throughput screening of novel electrolyte solvent combinations for next-generation Na-ion and Mg-ion batteries, where desolvation energetics are even more critical than in Li-ion systems [35,36].

Data Availability

Benchmark data (summary.csv, individual run logs) and validation input files are available at github.com/mirryou-maker/lammps-CO. All modified LAMMPS source files will be submitted as pull requests to the LAMMPS GitHub repository (github.com/lammps/lammps) upon journal acceptance.

Code Availability

All modified LAMMPS source files, benchmarking scripts (PowerShell harness and Python analysis tools), FEP input files, and the automated numerical verification pipeline are available at github.com/mirryou-maker/lammps-CO under the MIT licence. The modified pair-style source files will be submitted as pull requests to the LAMMPS `develop` branch at github.com/lammps/lammps upon journal acceptance.

Acknowledgements

The author thanks the LAMMPS developer community for maintaining extensive developer documentation (`Developer_write_openmp.rst`, `Developer_par_openmp.rst`) that served as the authoritative reference for all OpenMP transformations performed in this work. Computational resources were provided by DGIST Research Computing. This work was supported by the National Research Foundation of Korea (NRF) (No. RS-2026-25472340) of the Ministry of Science, ICT, and Future Planning.

Author Contributions

C.-Y.Y.: Conceptualization (study design and six-step protocol); Data curation (benchmark logs, thermo outputs, FEP window data); Formal analysis (speedup statistics, Amdahl-law fitting, TI/BAR free energy analysis); Investigation (all benchmarking experiments and FEP simulations); Methodology (development of the risk-stratified LLM-assisted optimization workflow and automated numerical verification pipeline); Project administration; Software (implementation of 36 new OMP pair-style variants, restrict/fxtmp backport to 225 serial hotloops, `pow()` deduplication in Steps A-4, benchmarking harness, FEP data-analysis scripts); Validation (bit-identical thermo comparison for every modified file, NVE energy conservation, TI/BAR cross-validation); Visualization (all six main figures and three supplementary figures); Writing — original draft (AI-assisted drafting and editing with Claude Code under author supervision); Writing — review & editing.

Competing Interests

The author declares no competing interests.

Generative AI Statement

Claude Code (Anthropic; model version `claude-sonnet-4-6`; accessed June 2025–July 2026) was used as an AI coding agent to generate, transform, and refactor C++ source files under the direct

supervision of the author. All AI-generated code was reviewed by the author and subjected to automated bit-identical numerical verification against serial reference implementations before incorporation into the codebase. The specific prompt templates and transformation rules used to guide the AI are provided in Supplementary Methods S1. Claude Code was additionally used to assist in drafting and editing portions of the manuscript text, under the direct supervision and critical review of the author; all final content reflects the author’s scientific judgement and has been verified for accuracy.

References

1. Thompson, A. P. *et al.* LAMMPS—A flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales. *Comput. Phys. Commun.* **271**, 108171 (2022). <https://doi.org/10.1016/j.cpc.2021.108171>
2. Plimpton, S. Fast parallel algorithms for short-range molecular dynamics. *J. Comput. Phys.* **117**, 1–19 (1995). <https://doi.org/10.1006/jcph.1995.1039>
3. Buehler, M. J. & Gao, H. Dynamical fracture instabilities due to local hyperelasticity at crack tips. *Nature* **439**, 307–310 (2006). <https://doi.org/10.1038/nature04408>
4. He, X. *et al.* Liquid-like dynamics in a solid-state lithium electrolyte. *Nat. Phys.* **20**, 1755–1761 (2024). <https://doi.org/10.1038/s41567-024-02707-6>
5. Famprikis, T., Canepa, P., Dawson, J. A., Islam, M. S. & Masquelier, C. Fundamentals of inorganic solid-state electrolytes for batteries. *Nat. Mater.* **18**, 1278–1291 (2019). <https://doi.org/10.1038/s41563-019-0431-3>
6. Batatia, I. *et al.* MACE: Higher-order equivariant message passing neural networks for fast and accurate force fields. *NeurIPS* **35**, 11423–11436 (2022). <https://arxiv.org/abs/2206.07697>
7. Erhard, L. C., Rohrer, J., Albe, K. & Deringer, V. L. Modelling atomic and nanoscale structure in the silicon–oxygen system through active machine learning. *Nat. Commun.* **15**, 1927 (2024). <https://doi.org/10.1038/s41467-024-45840-9>
8. Cheng, B., Engel, E. A., Behler, J., Dellago, C. & Ceriotti, M. Ab initio thermodynamics of liquid and solid water. *Proc. Natl Acad. Sci. USA* **116**, 1110–1115 (2019). <https://doi.org/10.1073/pnas.1815117116>
9. Bauer, B. A. & Patel, S. Recent applications and developments of charge equilibration force fields for modeling dynamical charges in classical molecular dynamics simulations. *Theor. Chem. Acc.* **131**, 1153 (2012). <https://doi.org/10.1007/s00214-012-1153-7>
10. Brown, W. M., Wang, P., Plimpton, S. J. & Tharrington, A. N. Implementing molecular dynamics on hybrid high performance computers — short range forces. *Comput. Phys. Commun.* **182**, 898–911 (2011). <https://doi.org/10.1016/j.cpc.2010.12.021>
11. Edwards, H. C., Trott, C. R. & Sunderland, D. Kokkos: Enabling manycore performance portability through polymorphic memory access patterns. *J. Parallel Distrib. Comput.* **74**, 3202–3216 (2014). <https://doi.org/10.1016/j.jpdc.2014.07.003>
12. Gao, S. *et al.* Unveiling the mechanisms of strength–ductility synergy in an additively manufactured nanolamellar high-entropy alloy. *Nat. Commun.* **16**, 5221 (2025). <https://pmc.ncbi.nlm.nih.gov/articles/PMC12606152/>
13. Gissinger, J. R., Zinchenko, A., Jensen, B. D. & Wise, K. E. Type label framework

- for bonded force fields in LAMMPS. *J. Phys. Chem. B* **128**, 3282–3297 (2024). <https://doi.org/10.1021/acs.jpcc.3c08419>
14. Gissinger, J. R. LUNAR: Automated input generation and analysis for reactive LAMMPS simulations. *J. Chem. Inf. Model.* **64**, 5998–6007 (2024). <https://doi.org/10.1021/acs.jcim.4c00730>
 15. Maresca, F. & Curtin, W. A. Mechanistic origin of high strength in refractory BCC high entropy alloys up to 1900 K. *Acta Mater.* **182**, 235–249 (2020). <https://doi.org/10.1016/j.actamat.2019.10.015>
 16. Ferreira, A. C. *et al.* Dynamic nanodomains dictate macroscopic properties in lead halide perovskites. *Nat. Nanotechnol.* **20**, 536–545 (2025). <https://arxiv.org/abs/2404.14598>
 17. Zhang, L. *et al.* A sodium superionic chloride electrolyte driven by paddle wheel mechanism for solid-state batteries. *Nat. Commun.* **16**, 3221 (2025). <https://www.researchgate.net/publication/393797071>
 18. Aktulga, H. M., Fogarty, J. C., Pandit, S. A. & Grama, A. Y. Parallel reactive molecular dynamics: Numerical methods and algorithmic techniques. *Parallel Comput.* **38**, 245–259 (2012). <https://doi.org/10.1016/j.parco.2011.08.005>
 19. Austin, J. *et al.* Program synthesis with large language models. arXiv:2108.07732 (2021). <https://arxiv.org/abs/2108.07732>
 20. Christ, C. D., Mark, A. E. & van Gunsteren, W. F. Basic ingredients of free energy calculations: A review. *J. Comput. Chem.* **31**, 1569–1582 (2010). <https://doi.org/10.1002/jcc.21450>
 21. You, C.-Y. Optimization of MuMax3 by using Claude Code: A CUDA-graph-based case study in AI-assisted performance engineering. *J. Magn.* **31**, 204–213 (2026). <https://doi.org/10.4283/JMAG.2026.31.2.204>
 22. Shirts, M. R. & Chodera, J. D. Statistically optimal analysis of samples from multiple equilibrium states. *J. Chem. Phys.* **129**, 124105 (2008). <https://doi.org/10.1063/1.2978177>
 23. Alvarado, F. J. Q. *et al.* Multiscale simulation of solid electrolyte interface formation in fluorinated diluted electrolytes with lithium anodes. *ACS Appl. Mater. Interfaces* **14**, 7060–7070 (2022). <https://doi.org/10.1021/acsami.1c22610>
 24. Nijkamp, E. *et al.* A conversational paradigm for program synthesis. arXiv:2203.13474 (2022). <https://arxiv.org/abs/2203.13474>
 25. Hermann, J., DiStasio, R. A. & Tkatchenko, A. First-principles models for van der Waals interactions in molecules and materials. *Chem. Rev.* **117**, 4714–4758 (2017). <https://doi.org/10.1021/acs.chemrev.6b00446>
 26. Lahkar, S. *et al.* LAMMPS-CTIP-EChemDID: Charge transfer interatomic potential implementation in LAMMPS. *GitHub* (2023). <https://github.com/simantalahkar/LAMMPS-CTIP-EChemDID>
 27. Tóth, M., Broqvist, P. & Kullgren, J. Extending Hamiltonian-adaptive resolution simulation to interfaces: An updated LAMMPS implementation and application to porous solids. arXiv:2604.21867 (2025). <https://arxiv.org/html/2604.21867v1>
 28. Jorgensen, W. L., Maxwell, D. S. & Tirado-Rives, J. Development and testing of the OPLS all-atom force field on conformational energetics and properties of organic liquids. *J. Am. Chem. Soc.* **118**, 11225–11236 (1996). <https://doi.org/10.1021/ja9621760>

29. Beutler, T. C., Mark, A. E., van Schaik, R. C., Greber, P. R. & van Gunsteren, W. F. Avoiding singularities and numerical instabilities in free energy calculations based on molecular simulations. *Chem. Phys. Lett.* **222**, 529–539 (1994). [https://doi.org/10.1016/0009-2614\(94\)00397-1](https://doi.org/10.1016/0009-2614(94)00397-1)
30. Bhati, A. P., Wan, S., Wright, D. W. & Coveney, P. V. Rapid, accurate, precise, and reliable relative free energy prediction using ensemble based thermodynamic integration. *J. Chem. Theory Comput.* **13**, 210–222 (2017). <https://doi.org/10.1021/acs.jctc.6b00979>
31. Chen, M. *et al.* Evaluating large language models trained on code. *arXiv:2107.03374* (2021). <https://arxiv.org/abs/2107.03374>
32. Gapsys, V. *et al.* Large scale relative protein ligand binding affinities using non-equilibrium alchemy. *Chem. Sci.* **11**, 1140–1152 (2020). <https://doi.org/10.1039/C9SC03754C>
33. Joung, I. S. & Cheatham, T. E. III. Determination of alkali and halide monovalent ion parameters for use in explicitly solvated biomolecular simulations. *J. Phys. Chem. B* **112**, 9020–9041 (2008). <https://doi.org/10.1021/jp8001614>
34. Xu, K. Nonaqueous liquid electrolytes for lithium-based rechargeable batteries. *Chem. Rev.* **104**, 4303–4418 (2004). <https://doi.org/10.1021/cr030203g>
35. Xu, K. Electrolytes and interphases in Li-ion batteries and beyond. *Chem. Rev.* **114**, 11503–11618 (2014). <https://doi.org/10.1021/cr500003w>
36. Bhatt, M. D. & O’Dwyer, C. Recent progress in theoretical and computational investigations of Li-ion battery materials and electrolytes. *Phys. Chem. Chem. Phys.* **17**, 4799–4844 (2015). <https://doi.org/10.1039/C4CP05552G>
37. Wan, L.-F., Persson, K. A. *et al.* Lithium ion solvation by ethylene carbonates in lithium-ion battery electrolytes, revisited by density functional theory with the hybrid solvation model and free energy correction in solution. *Phys. Chem. Chem. Phys.* **18**, 17326–17335 (2016). <https://doi.org/10.1039/c6cp01667g>
38. Takeuchi, M. *et al.* Free-energy and structural analysis of ion solvation and contact ion-pair formation of Li^+ with BF_4^- and PF_6^- in water and carbonate solvents. *J. Phys. Chem. B* **116**, 6476–6487 (2012). <https://doi.org/10.1021/jp3011487>
39. Chaban, V. Solvation of lithium ion in dimethoxyethane and propylene carbonate. *Chem. Phys. Lett.* **625**, 110–115 (2015). <https://doi.org/10.1016/j.cplett.2015.02.065>
40. Pirani, F.; Alberti, M.; Castro, A.; Teixidor, M. M. & Cappelletti, D. Atom-bond pairwise additive representation for intermolecular potential energy surfaces. *Chem. Phys. Lett.* **394**, 37–44 (2004). <https://doi.org/10.1016/j.cplett.2004.06.100>

Supplementary Materials

Supplementary Table S1: Complete list of all 36 newly added OMP pair variants with source files and physical descriptions.

Supplementary Table S2: Complete A-3 modification log: all 225 modified pair files with before/after verification status.

Supplementary Methods: Full Claude Code prompt templates for OMP port generation (half-list, full-list/DIELECTRIC, and KSPACE variants) and A-3 restrict/fxtmp backporting.

Supplementary Figure S1: Full OMP scaling benchmarks for all three newly parallelized styles at 1, 2, 4, and 8 threads.

Supplementary Figure S2: NVE energy conservation traces (total energy vs. step) comparing serial and OMP-4-thread execution for `born/coul/dsf` and `lj/cut/soft`, confirming thermodynamic equivalence.

Supplementary Figure S3: A-3 code transformation (before/after pair hotloop): `__restrict__/_noalias` pointer annotation and `fxtmp/fytmp/fztmp` local-accumulator pattern applied to standard serial pair files.

Manuscript submitted to Archives of Computational Methods in Engineering